

6장. 분산 키값 저장소

문주호

key-value store

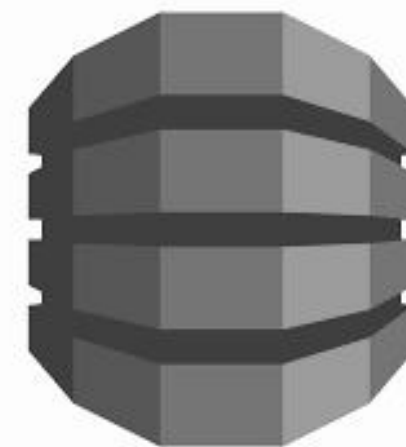
distributed key-value store

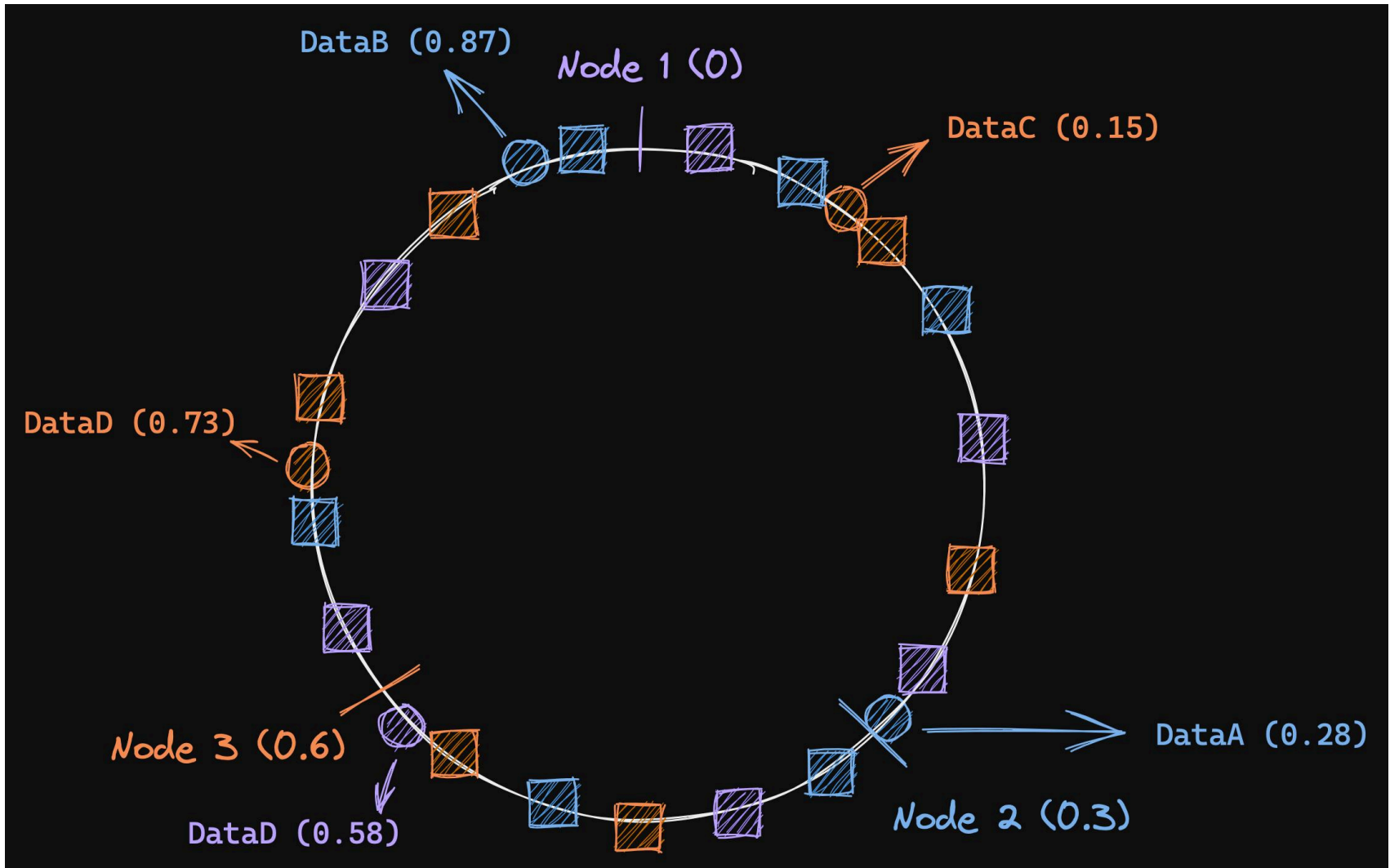
CAP theorem

cp system



분산 키값 저장소





데이터 파티션

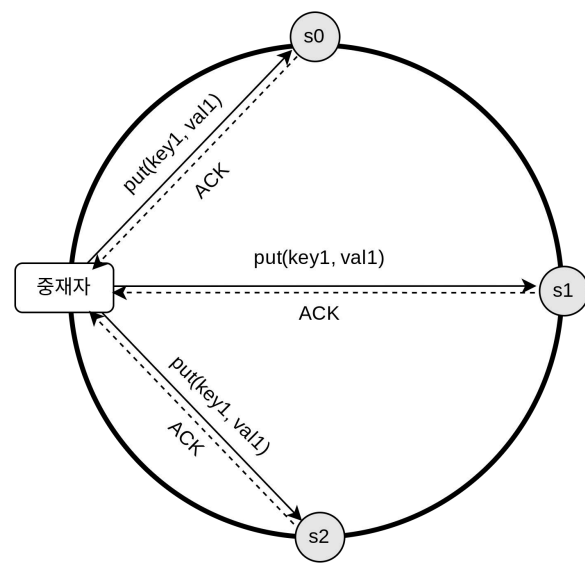
- hash ring 개념은 5장때 진행했으니 넘어감

데이터 다중화

- 데이터 다중화 시 같은 데이터 센터 노드를 사용하지 않도록 만들어야함

데이터 일관성

- N : 사본 개수
- W : 쓰기 연산에 대한 정족수. 쓰기 연산이 성공하려면 적어도 W개의 서버로부터 성공 응답을 받아야 함
- R : 읽기 연산에 대한 정족수. 읽기 연산이 성공하려면 적어도 W개의 서버로부터 성공 응답을 받아야 함



데이터 일관성

- 강한 일관성(Strong Consistency) : 모든 읽기 연산은 가장 최근에 갱신된 결과를 반환한다. 즉 클라이언트는 절대로 낡은(out-of-date) 데이터를 보지 못한다.
- 약한 일관성(Weak Consistency) : 읽기 연산은 가장 최근에 갱신된 결과를 반환하지 못할 수 있다.
- 최종 일관성/결과적 일관성(Eventual Consistency) : 약한 일관성의 한 형태로, 갱신 결과가 결국에는 모든 사본에 반영(즉, 동기화)되는 모델이다.

정족수 합의 프로토콜

N,W,R 값에 따른 구성

- R = 1, W = N: 빠른 읽기 연산에 최적화된 시스템
- W = 1, R = N: 빠른 쓰기 연산에 최적화된 시스템
- W + R > N: 강한 일관성이 보장됨 (보통 N=3, W=R=2)
- W + R ≤ N: 강한 일관성이 보장되지 않음

데이터 버저닝

데이터를 다중화 하면 가용성은 높아지지만 사본 간 일관성이 깨질 가능성은 높아진다.

버저닝(versioning)과 벡터 시계(vector clock)는 그 문제를 해소하기 위해 등장한 기술이다.

동일하게 name: john 이라는 값을 가지고 있는 서버 n1, n2 에서

동시에 n1 에서는 “johnSanFrancisco”로, n2 에서는 “johnNewyork”으로 바꾼다고 하자.

이렇게 되면 충돌(conflict)하는 두 값을 갖게 된다. 그 각각을 버전 v1, v2 라고 하자.

벡터 시계는 [서버, 버전]의 순서쌍을 데이터에 매단 것이다. 어떤 버전이 선행 버전인지, 후행 버전인지, 아니면 다른 버전과 충돌이 있는지 판별하는데 쓰인다.

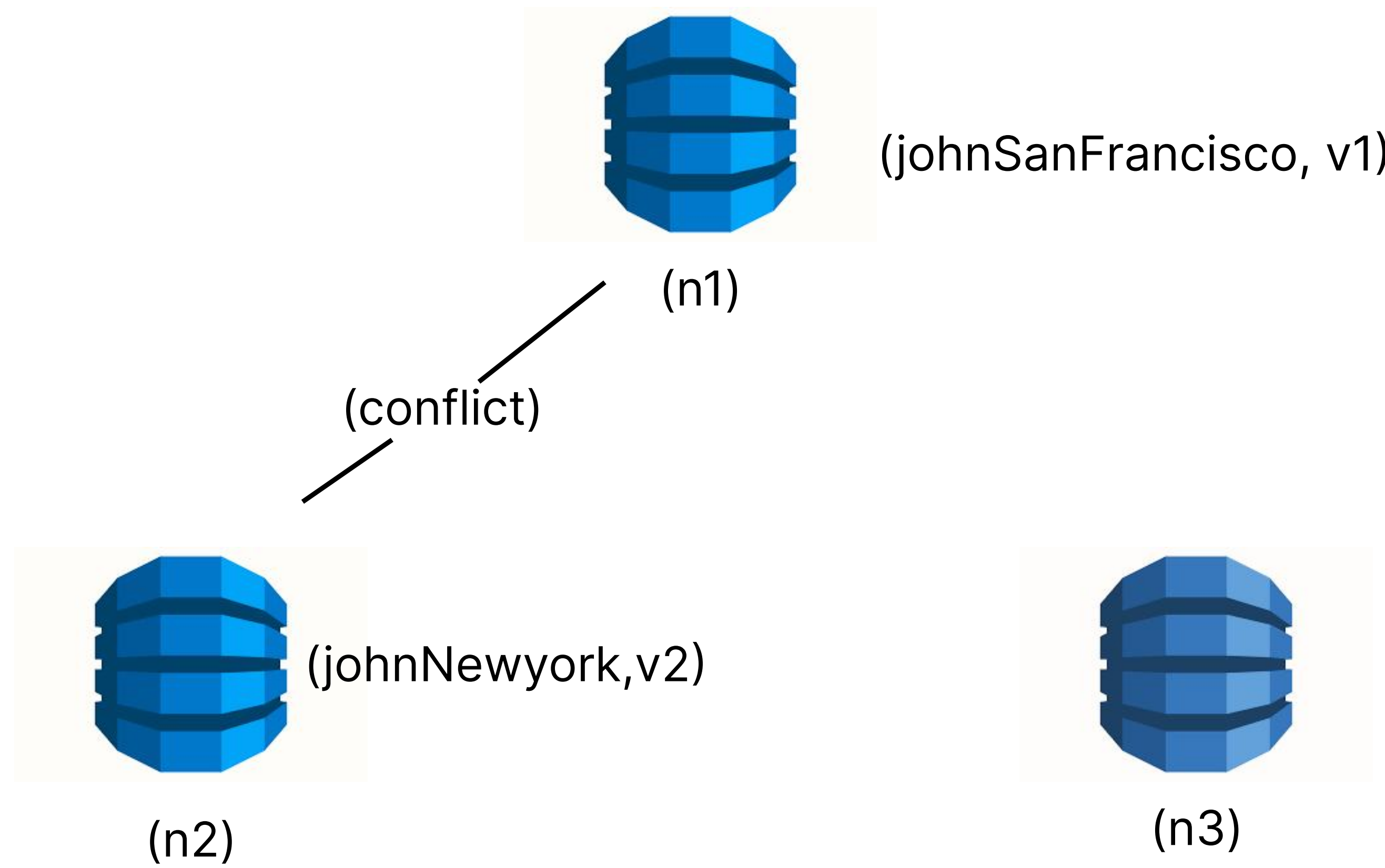
벡터 시계는 D([S1, v1], [S2, v2], ... [Sn, vn])와 같이 표현한다고 가정하자.

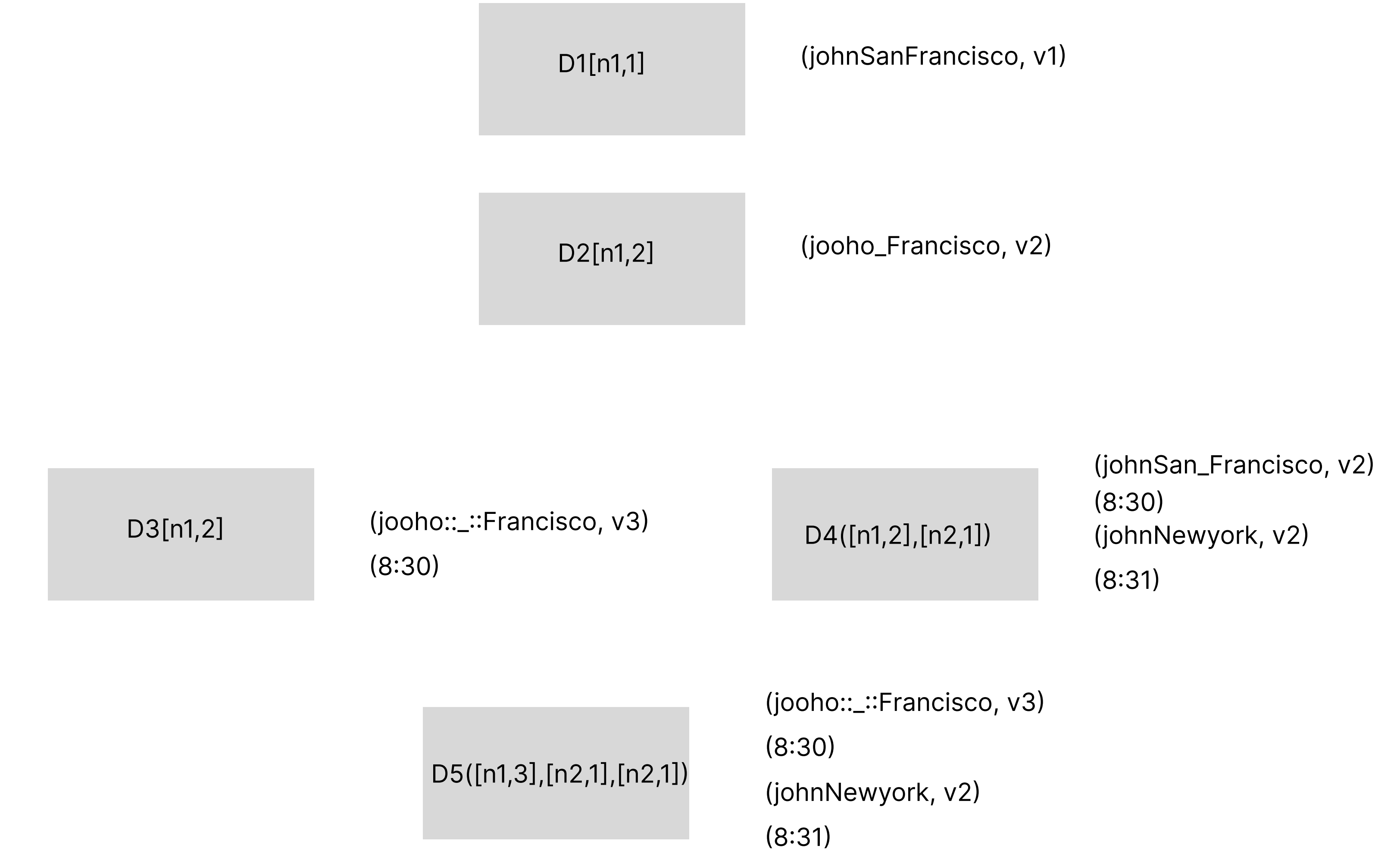
여기서 D는 데이터이고, vi 는 버전 카운터 Si는 서버 번호 이다.

만일 데이터 D를 서버 Si에 기록하면, 시스템은 아래 작업 가운데 하나를 수행하여야 한다.

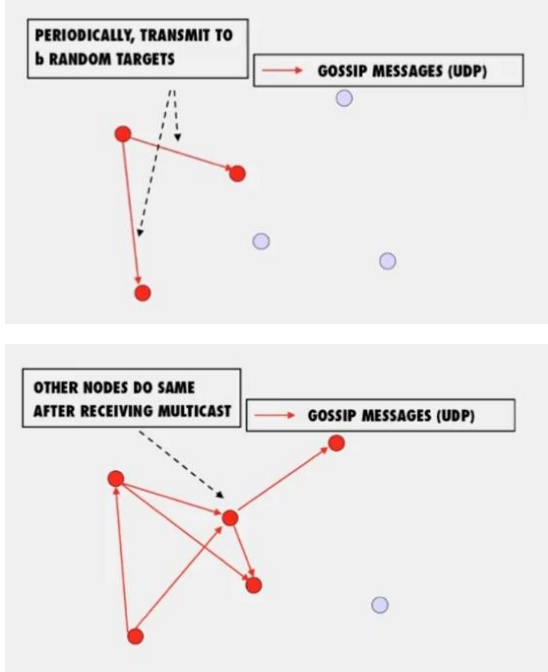
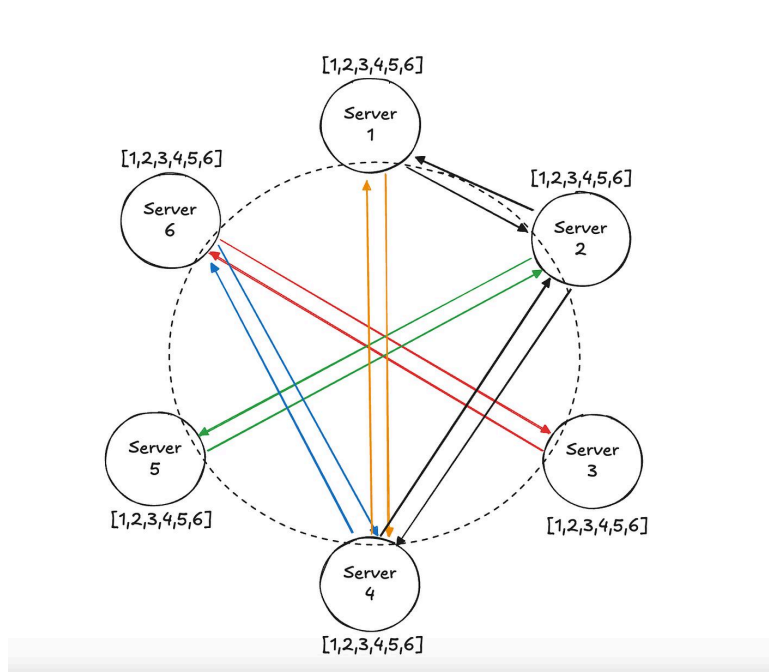
- [Si, vi]가 있으면 vi를 증가시킨다.
- 그렇지 않으면 새 항목 [Si, 1]를 만든다..

데이터 버저닝





전략 명칭	핵심 메커니즘	장점	단점
LWW	시스템 시각(Timestamp) 비교	속도가 빠르고 단순함	시계 불일치 시 데이터 유실 가능성
Vector Clock	논리적 인과 관계 추적	정확한 충돌 감지 및 보존	메타데이터 크기가 커짐
CRDT	특수 데이터 구조 (Set, Counter)	자동으로 값이 합쳐짐(Merge)	설계가 복잡하고 제약이 많음



장애 감지

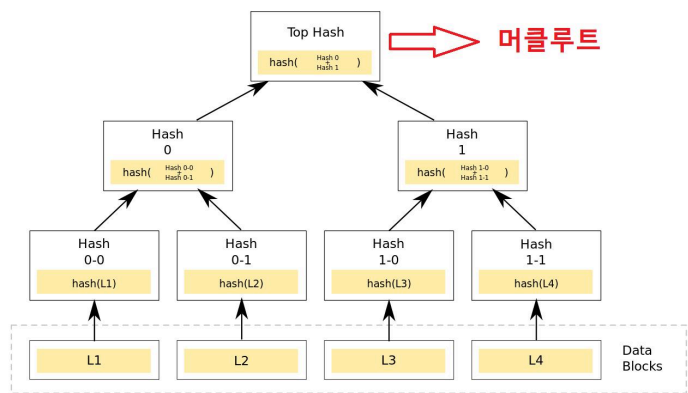
분산 시스템에서는 그저 한 대 서버가 “지금 서버 A가 죽었습니다”라고 한다 해서 바로 서버 A를 장애처리 하지는 않는다. 보통 두 대 이상의 서버가 똑같이 서버 A의 장애를 보고해야 해당 서버에 실제로 장애가 발생했다고 간주하게 된다. 모든 노드 사이에 멀티캐스팅(multicasting) 채널을 구축하는 것이 서버 장애를 감지하는 가장 쉬운 방법이다. 하지만 이 방법은 서버가 많을 때는 분명 비효율 적이다.

* 멀티캐스트(multicast)란 한 번의 송신으로 메시지나 정보를 목표한 여러 컴퓨터에 동시에 전송하는 것을 말한다. (wiki)

가십 프로토콜(gossip protocol) 같은 분산형 장애 감지(decentralized failure detection) 솔루션을 채택하는 편이 보다 효율적이다.

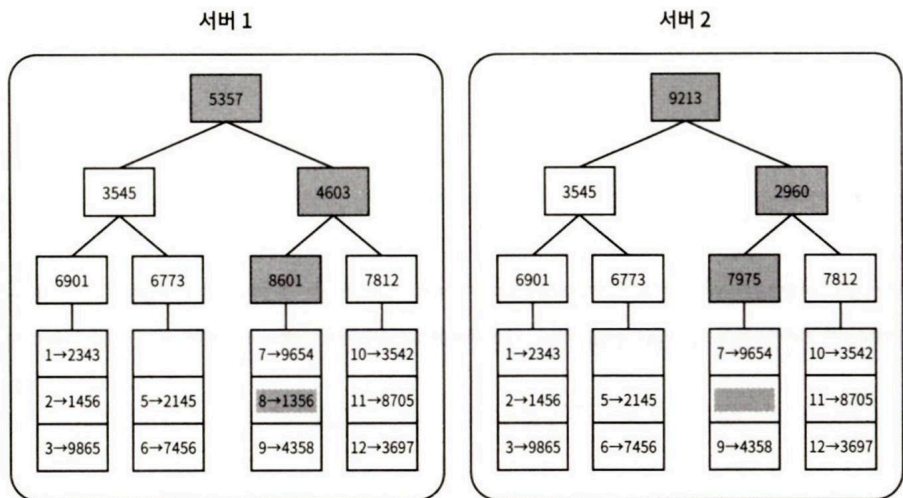
가십 프로토콜의 동작 원리는 다음과 같다.

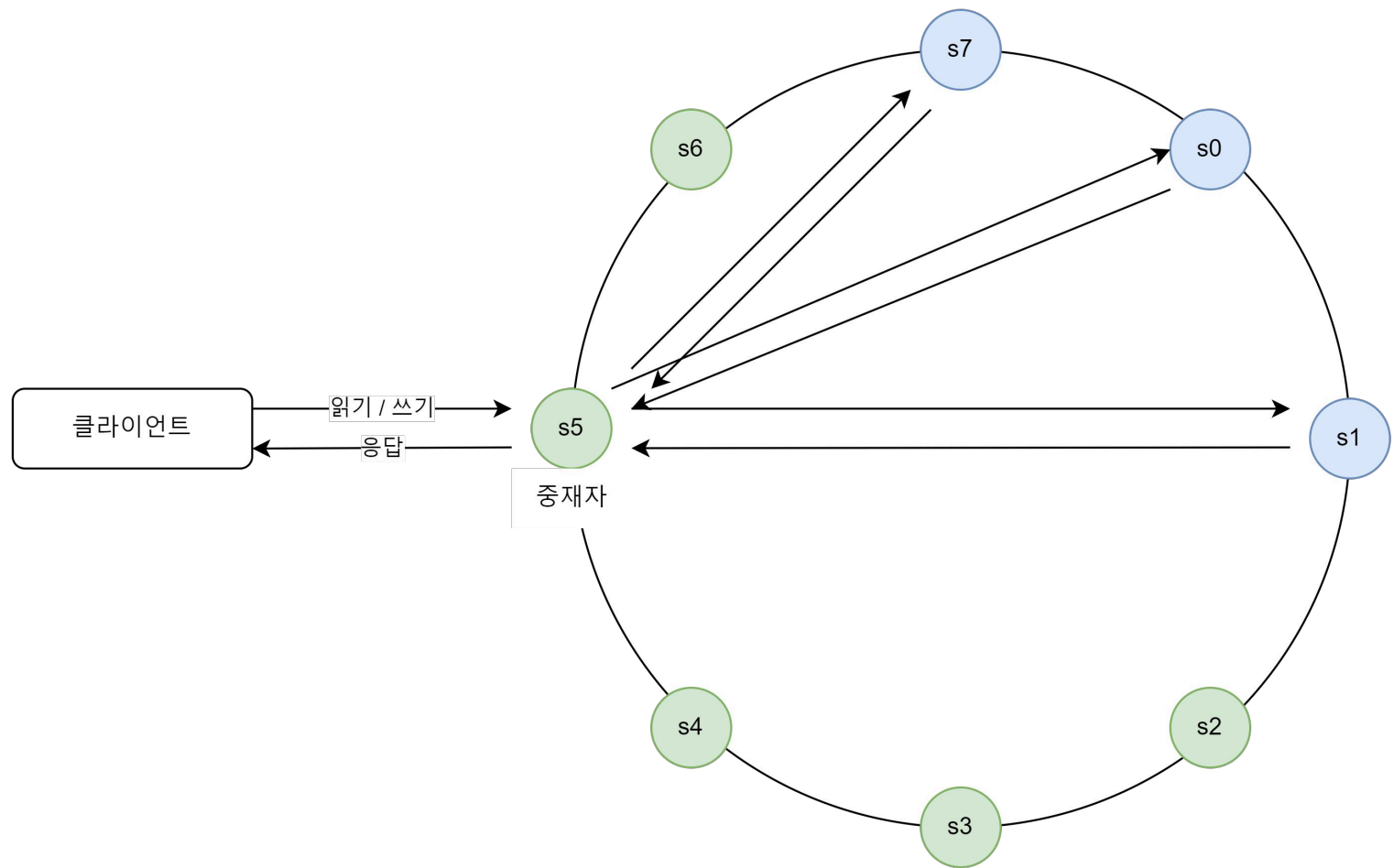
- 각 노드는 멤버십 목록(membership list)을 유지한다. 멤버십 목록은 각 멤버 ID와 그 박동 카운터(heartbeat counter) 쌍의 목록이다.
- 각 노드는 주기적으로 자신의 박동 카운터를 증가시킨다.
- 각 노드는 무작위로 선정된 노드들에게 주기적으로 자기 박동 카운터 목록을 보낸다.
- 박동 카운터 목록을 받은 노드는 멤버십 목록을 최신 값으로 갱신한다.
- 어떤 멤버의 박동 카운터 값이 지정된 시간 동안 갱신되지 않으면 해당 멤버는 장애(offline) 상태인 것으로 간주한다.



우리 노드의 장애가 영구적이 될 수 있는데, 이럴 때는 반-엔트로피 프로토콜을 구현해서 사본들을 동기화 할 것이다. 머클 트리를 이용하여 사본 간의 일관성이 망가진 상태를 탐지하고 전송 데이터 양을 줄인다.

- 1단계: 키 공간을 버킷으로 나눔 (예제는 4개)
- 2단계: 버킷에 포함된 각각의 키에 균등 분포 해시 함수를 적용하여 해시 값을 계산한다.
- 3단계: 버킷별로 해시값을 계산한 후, 해당 해시 값을 레이블로 갖는 노드를 만든다.
- 4단계: 자식 노드의 레이블로부터 새로운 해시값을 계산하여 이진트리를 상향식으로 구성해나간다.



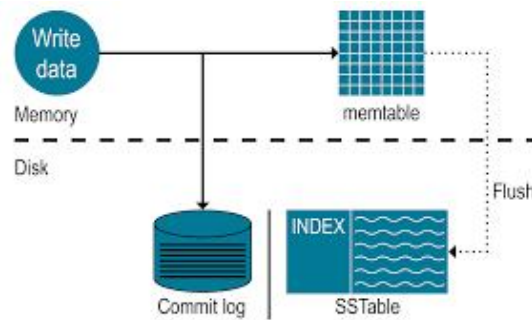


- 클라이언트는 키-값 저장소가 제공하는 API get(key)와 put(key, value)로 통신한다.
- 중재자는 클라이언트에게 키-값 저장소에 대한 프락시 역할을 하는 노드다.
- 노드는 안정해시의 해시링 위에 분포한다.
- 노드를 자동으로 추가 또는 삭제할 수 있도록, 시스템은 완전히 분산된다.
- 데이터는 여러 노드에 다중화된다.
- 모든 노드가 같은 책임을 지므로, SPOF(Single Point of Failure)는 존재하지 않는다.

쓰기 경로(ex. 카산드라)

1. 쓰기 요청이 커밋 로그 파일에 기록된다.
2. 데이터가 메모리 캐시에 기록된다.
3. 메모리 캐시가 가득차거나 사전에 정의된 어떤 임계치에 도달하면 데이터는 디스크에 있는 SSTable에 기록된다.

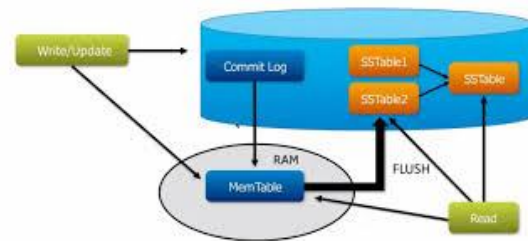
SSTable은 Sorted-String Table의 약어로, <키, 값>의 순서쌍을 정렬된 리스트 형태로 관리하는 테이블이다.



읽기 경로

읽기 요청을 받은 노드는 데이터가 메모리 캐시에 있는지부터 살핀다. 있는 경우에는 해당 데이터를 클라이언트에게 반환한다. 데이터가 메모리에 없는 경우에는 디스크에서 가져와야한다. 어느 SSTable에 키가 있는지 알아낼 방법으로 bloom 필터가 흔히 사용된

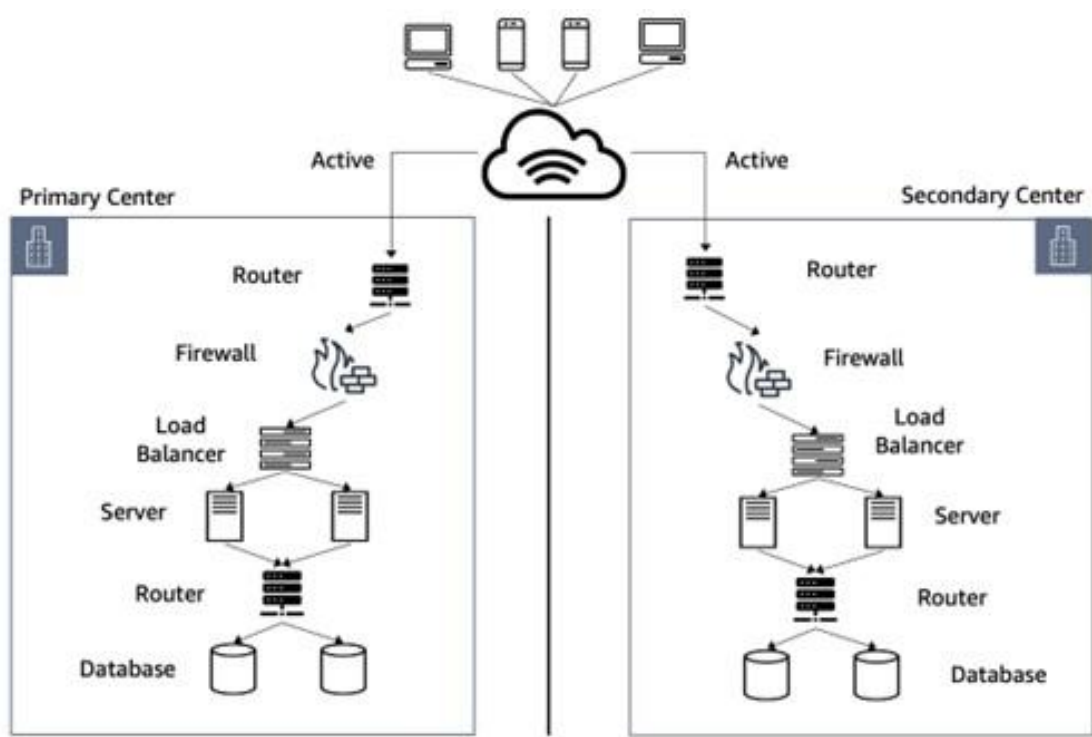
1. 데이터가 메모리 있는지 검사한다. 없으면 2로 간다.
2. 데이터가 메모리에 없으므로 bloom 필터를 검사한다.
3. bloom 필터를 통해 어떤 SSTable에 키가 보관되어 있는지 알아낸다.
4. SSTable에서 데이터를 가져온다.
5. 해당 데이터를 클라이언트에게 반환한다.



데이터 센터 장애 처리

데이터 센터 장애는 정전, 네트워크 장애, 자연재해 등 다양한 이유로 발생
어떨땐 인프라 구조의 이유로 장애가 발생 하기도 한다.

이를 해결하기 위해선 보통 DR



감사합니다